

```

import sys
import platform

# --- 1. AUTO-DETECT ENVIRONMENT ---
# Check if we are in Termux (Android)
if "com.termux" in os.environ.get("PREFIX", ""):
    DEVICE_NAME = "Pixel 8a (Mobile)"
    ROOT_DIR = "/storage/emulated/0/pixel8a/unexusi/"
else:
    DEVICE_NAME = "Laptop (Base)"
    ROOT_DIR = "/home/sauron/unexusi/"

# --- IMPORTS ---
try:
    from git import Repo
except ImportError:
    print(f"❌ Critical Error on {DEVICE_NAME}: 'GitPython' is not installed.")
    sys.exit()

# --- CONFIGURATION ---
DEFAULT_IGNORE = """
__pycache__/
*.pyc
.DS_Store
.vscode/
env/
venv/
"""

def check_permissions():
    """Safety check for storage permissions and directory existence"""
    if not os.path.exists(ROOT_DIR):
        print(f"⚠️ WARNING: Cannot find directory: {ROOT_DIR}")
        print(f"Current Mode: {DEVICE_NAME}")
        if "Mobile" in DEVICE_NAME:
            print("1. Check if the folder exists.")
            print("2. Run: termux-setup-storage")
        return False
    return True

def setup_alias():
    """Injects the 'fleet' alias into .bashrc automatically"""
    bashrc_path = os.path.expanduser("~/bashrc")
    script_path = os.path.abspath(__file__)

```

```
# The exact line we want to add
alias_cmd = f'alias fleet="python {script_path}"

print(f"\n🔧 Configuring Alias for {DEVICE_NAME}...")
```

```
# Check if already exists
try:
    with open(bashrc_path, "r") as f:
        if alias_cmd in f.read():
            print("✅ 'fleet' alias is already active!")
            return
```

```
except FileNotFoundError:
    pass # File doesn't exist yet, we will create it
```

```
# Append to file
with open(bashrc_path, "a") as f:
    f.write(f"\n{alias_cmd}\n")
```

```
print(" ✨ Success! Alias added.")
print(" ⚠️ Please restart your terminal (or type 'source ~/.bashrc') to use it.")
```

```
def create_gitignore(repo_path):
    """Creates a default .gitignore if missing"""
    ignore_path = os.path.join(repo_path, ".gitignore")
    if not os.path.exists(ignore_path):
        with open(ignore_path, "w") as f:
            f.write(DEFAULT_IGNORE)
        print(" ✨ Created default .gitignore")
    else:
        print(" ⚠️ .gitignore already exists.")
```

```
def get_git_status(repo_path):
    """Returns a status dictionary for a single repo."""
    try:
        repo = Repo(repo_path)
        status = {
            "path": repo_path,
            "name": os.path.basename(repo_path),
            "dirty": False,
            "ahead": 0,
            "behind": 0,
            "repo": repo,
            "branch": "unknown"
```

```

}

# Get Branch
try:
    status["branch"] = repo.active_branch.name
except:
    status["branch"] = "DETACHED"

# 1. Check Local Dirty
if repo.is_dirty(untracked_files=True):
    status["dirty"] = True

# 2. Check Sync (Fetch hiddenly)
try:
    origin = repo.remotes.origin
    origin.fetch()
    local_branch = repo.active_branch
    remote_branch = local_branch.tracking_branch()

    if remote_branch:
        status["behind"] = sum(1 for c in
repo.iter_commits(f'{local_branch.name}..{remote_branch.name}'))
        status["ahead"] = sum(1 for c in
repo.iter_commits(f'{remote_branch.name}..{local_branch.name}'))
    except:
        pass
    return status
except:
    return None

def show_file_details(repo):
    """Lists specifically which files are changed"""
    print(f"\n 📄 File Status for [{os.path.basename(repo.working_dir)}]:")

    # Changed files
    for item in repo.index.diff(None):
        print(f" ✏ Modified: {item.a_path}")

    # Untracked files
    for item in repo.untracked_files:
        print(f" 🆕 New File: {item}")

    print("")

```

```

def sync_repo(repo_data, auto_message="Auto-sync via Fleet Commander"):
    """Helper to sync a single repo"""
    print(f" 🛠️ Processing: {repo_data['name']}...")
    repo = repo_data['repo']

    # Save
    if repo_data['dirty']:
        show_file_details(repo) # Show what we are committing
        repo.git.add(all=True)
        repo.index.commit(auto_message)
        print(" ✅ Local changes saved.")

    # Push/Pull
    try:
        origin = repo.remotes.origin
        if repo_data['behind'] > 0:
            origin.pull()
            print(" ✅ Pulled down new changes.")
        if repo_data['dirty'] or repo_data['ahead'] > 0:
            origin.push()
            print(" ✅ Pushed up to Cloud.")

        if not repo_data['dirty'] and repo_data['ahead'] == 0 and repo_data['behind'] == 0:
            print(" ✨ Already clean.")

    except Exception as e:
        print(f" ❌ Error syncing: {e}")

def main_dashboard():
    if not check_permissions():
        input("Press Enter to exit...")
        return

    while True:
        os.system('clear')
        print(f"\n🌐 --- FLEET COMMANDER: {DEVICE_NAME} ---")
        print(f"Scanning Sector: {ROOT_DIR}\n")

        repos_found = []

        # SCAN
        try:
            if os.path.exists(ROOT_DIR):
                folder_list = sorted(os.listdir(ROOT_DIR)) # Sort alphabetically

```

```

for folder_name in folder_list:
    folder_path = os.path.join(ROOT_DIR, folder_name)
    if os.path.isdir(folder_path) and os.path.isdir(os.path.join(folder_path, ".git")):
        stat = get_git_status(folder_path)
        if stat:
            repos_found.append(stat)

    # Status Logic
    icon = "✅"
    msg = "Synced"
    if stat["dirty"]:
        icon = "⚠️"
        msg = "Unsaved Work"
    elif stat["ahead"] > 0:
        icon = "⬆️"
        msg = f"Ahead (+{stat['ahead']})"
    elif stat["behind"] > 0:
        icon = "⬇️"
        msg = f"Behind (-{stat['behind']})"

    print(f" {len(repos_found)}. {icon} {stat['name']:<18} [{stat['branch']}] | {msg}")
except OSError as e:
    print(f"❌ Error scanning: {e}")

if not repos_found:
    print(f"No repositories found in {ROOT_DIR}")

print("\n-----")
print(" A. Sync ALL repositories")
print(" S. Setup Tools (Alias & .gitignore)")
print(" R. Refresh")
print(" Q. Quit")

choice = input("\nCommand >> ").lower()

if choice == 'q':
    break
elif choice == 'r':
    continue
elif choice == 's':
    print("\n🔧 SYSTEM SETUP")
    print(" 1. Install 'fleet' alias")
    print(" 2. Generate .gitignore for a repo")
    sub = input("   Select >> ")

```

```

if sub == '1':
    setup_alias()
    input(" Press Enter...")
elif sub == '2':
    idx = int(input(f" Repo Number (1-{len(repos_found)}): ")) - 1
    if 0 <= idx < len(repos_found):
        create_gitignore(repos_found[idx]['path'])
        input(" Press Enter...")

elif choice == 'a':
    print("\n🚀 STARTING BATCH SYNC...")
    for repo_data in repos_found:
        sync_repo(repo_data)
    input("\nBatch complete! Press Enter...")
elif choice.isdigit():
    idx = int(choice) - 1
    if 0 <= idx < len(repos_found):
        target = repos_found[idx]
        print(f"\n🚀 Managing: {target['name']}")

        if target['dirty']:
            show_file_details(target['repo'])

        if input(" Sync this repo? (y/n): ").lower() == 'y':
            sync_repo(target, auto_message=input(" Enter commit message (or Enter for
auto): ") or "Update via Fleet Commander")

        input("\nDone. Press Enter...")

if __name__ == "__main__":
    #adding history option
    def show_git_log(repo_path):
        """Shows the last 5 commits (Time Travel View)"""
        try:
            repo = Repo(repo_path)
            print(f"\n 📖 History for {os.path.basename(repo_path)}:")

            # Get last 5 commits
            commits = list(repo.iter_commits('master', max_count=5))

            for commit in commits:
                # Format time (e.g., "2 days ago" or date)
                commit_date = time.strftime("%Y-%m-%d %H:%M",
time.localtime(commit.committed_date))

```

```
print(f"      ♦ [{commit_date}] {commit.message.strip()} ({commit.author.name})")
```

```
except Exception as e:
```

```
    print(f"      ✖ Could not read history: {e}")
```

```
#adding L option
```

```
# ... inside main_dashboard loop ...
```

```
    print(" A. Sync ALL repositories")
```

```
    print(" S. Setup Tools")
```

```
    print(" L. View History (Time Travel)" # <--- NEW
```

```
    print(" R. Refresh")
```

```
    print(" Q. Quit")
```

```
choice = input("\nCommand >> ").lower()
```

```
# ... existing checks ...
```

```
elif choice == 'l': # <--- NEW BLOCK
```

```
    idx = int(input(f" Repo Number (1-{len(repos_found)}): ") - 1
```

```
    if 0 <= idx < len(repos_found):
```

```
        show_git_log(repos_found[idx]['path'])
```

```
    input(" Press Enter...")
```

```
main_dashboard()
```